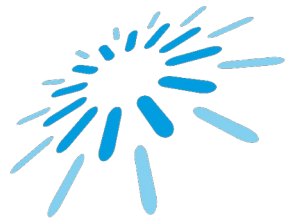




Software Architecture and Design Patterns

Prof. Dr. Jóakim von Kistowski

Lecture 07



TH Aschaffenburg
university of applied sciences

	Monday	Tuesday	Wednesday	Thursday	Friday
08:00 - 09:30					Vorlesung Software Design
09:45 - 11:15					Übung Software Design
11:30 - 13:00			Lecture Software Design International		
	Lunch Break				
14:00 - 15:30			Exercises Software Design International		
15:45 - 17:15					
17:30 - 19:00					

What are GRASP patterns?

What is the KISS principle?

Design Patterns

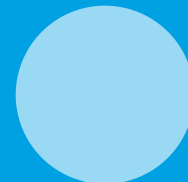


Patterns Revisited

Selected Gang of Four (GoF) Design Patterns

Anti-Pattners

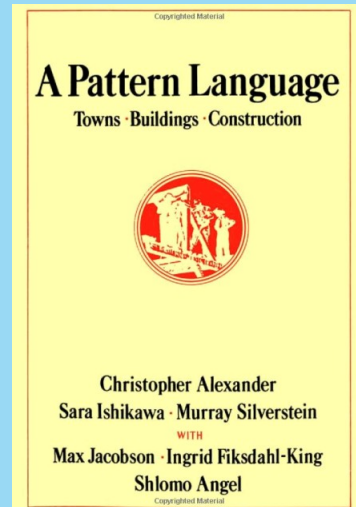
Concluding Remarks



Historical Perspective: Origin of Patterns

The basic idea was coined by Christopher Alexander in the context of buildings and towns:

“Each **pattern** describes a **problem which occurs over and over again** in our environment, and then **describes the core of the solution** to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

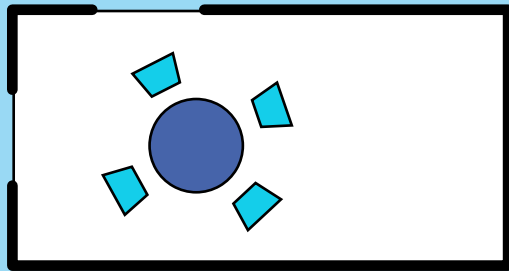


Christopher Alexander et al:
A Pattern Language,
Oxford University Press, New
York, 1977

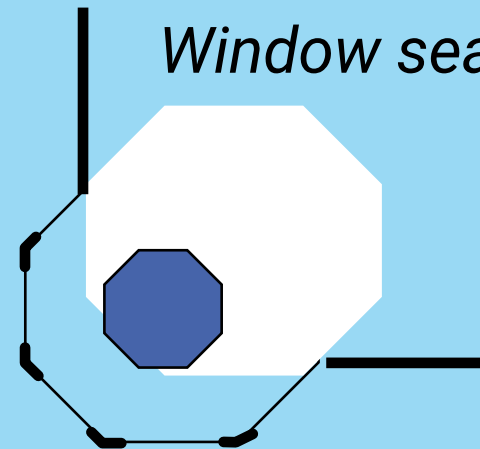
Building Architecture Patterns

Some example patterns as described by C. Alexander

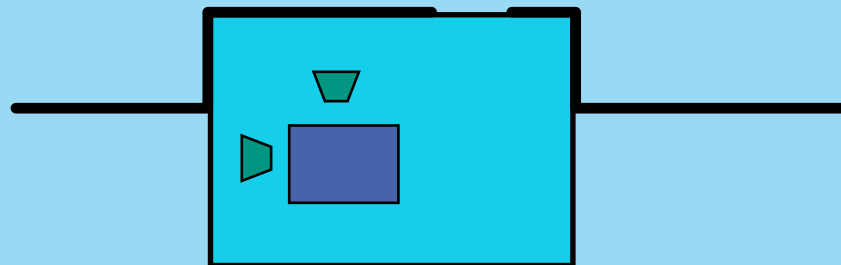
Light from two sides



Window seat



Deep Balcony



Two Sides ...

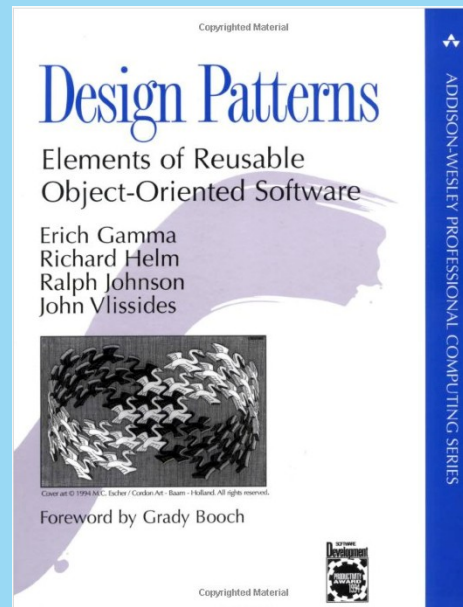
- Applying a pattern may involve a tradeoff between:
 1. **benefits**, as patterns –
 - help a designer get a design “right” faster
 - make it easier to reuse successful designs
 - help in building systems that are easy to maintain
 - improve the system documentation and understandability
 - make good designs available to more engineers
 2. and **costs**, as patterns may –
 - lower a system’s efficiency (and speed)
 - require more classes
 - require more run-time objects
- However, the central problem in designing (enterprise) systems today is dealing with complexity
 - not with resource limitations anymore

Essential Elements of a Pattern

- **NAME**
 - a handle for identifying a design problem, its solution, and consequences
 - increases design vocabulary
 - makes it easier to discuss designs on a high level
- **PROBLEM**
 - places where the pattern is applicable
 - may include a list of conditions for the beneficial use of the pattern
- **SOLUTION**
 - the elements that make up the design, their relationships, responsibilities and interactions
 - no particular solution, only an abstract design
- **CONSEQUENCES**
 - the results and tradeoffs of applying the pattern (pros and cons)
 - critical for evaluating design decisions

Gang of Four Design Patterns

- Design patterns are immutably connected with the famous book by the Gang of Four (GoF)
 - Organization of patterns in a catalogue-like style
 - Pattern description according to a template
 - Categorization



*GoF = Erich Gamma + Richard Helm +
Ralph Johnson + John Vlissides*

Description Template

- Pattern Name and Classification
- Intent
- Also Known As
- Motivation
- Applicability
- Structure
- Participants
- Collaborations
- Consequences
- Implementation
- Sample Code
- Known Uses
- Related Patterns

The GoF Patterns can be classified

... in a two-dimensional space as follows

1. According to their **purpose**, which can be –
 - **Creational** (related to the creation of objects)
 - **Structural** (concerned with structures/compositions)
 - **Behavioral** (influences how classes and objects collaborate)
2. According to their **scope**, which can be either of –
 - **class** (concerned with classes and their subclasses)
 - relationships are arranged by inheritance (thus they are static)
 - **object** (concerned with the relations of objects...)
 - ...that may be changed at run-time, thus they are dynamic

GoF: Design Pattern Space

Purpose Scope	Creational	Structural	Behavioral
Class	Factory Method	Adapter (class)	Interpreter Template Method
Object	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Relationship between GoF and GRASP

- Most OO design patterns, including GoF patterns, can be seen as specializations (special cases) of the nine basic GRASP principles
- **General Responsibility Assignment Software Patterns (GRASP)**
 1. Information Expert or just “Expert”
 2. Creator
 3. Controller
 4. Low Coupling
 5. High Cohesion
 6. Polymorphism
 7. Pure Fabrication
 8. Indirection
 9. Protected Variations

Design Patterns

1. Adapter
2. Factory
3. Singleton
4. Strategy
5. Composite
6. Observer

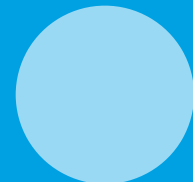


Patterns Revisited

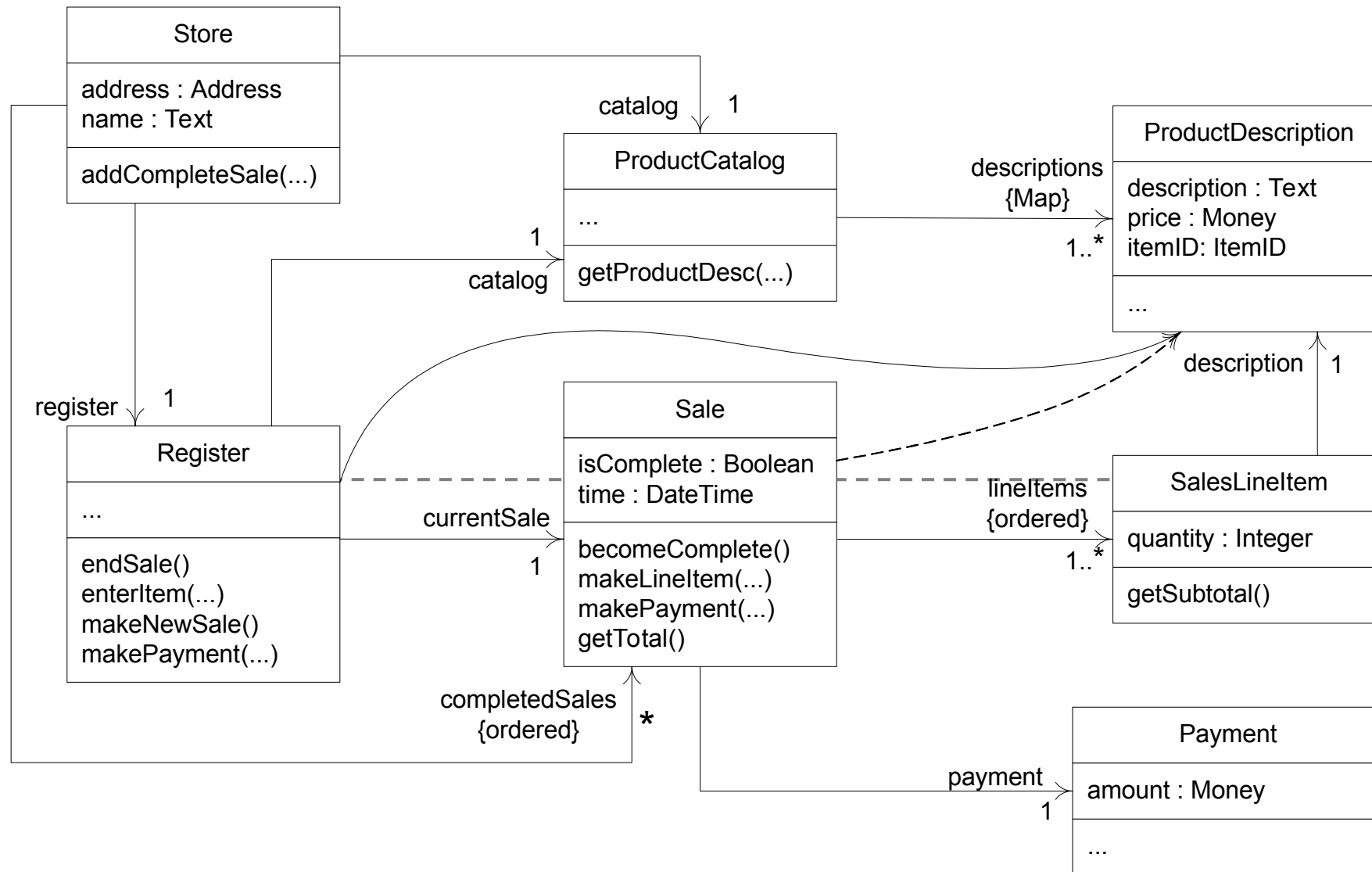
Selected Gang of Four (GoF) Design Patterns

Anti-Pattners

Concluding Remarks

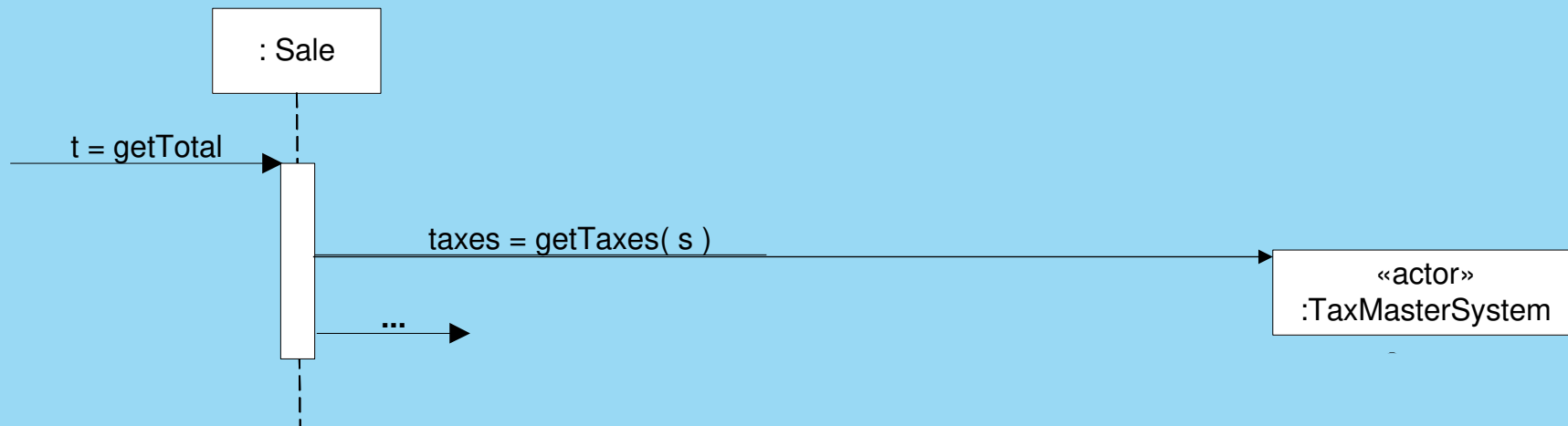


Refresh: Example POS System



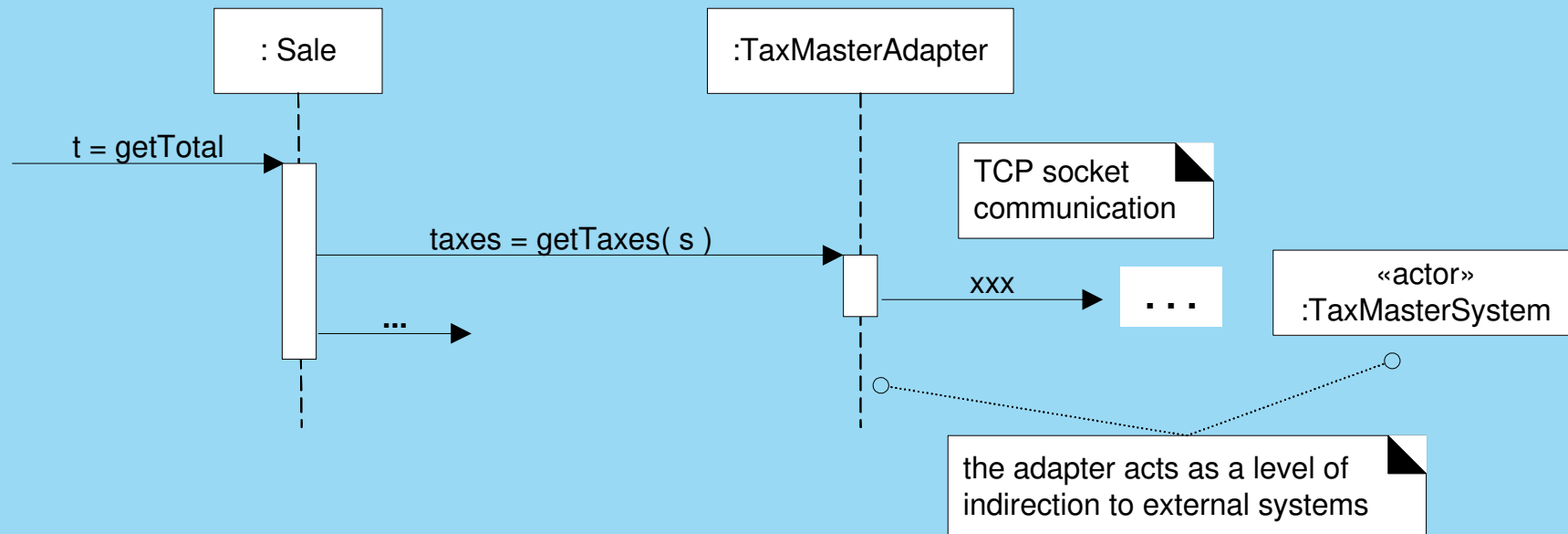
1. Adapter Pattern (GoF)

- **Problem:** How to resolve incompatible interfaces, or provide a stable interface to similar components with different interfaces?

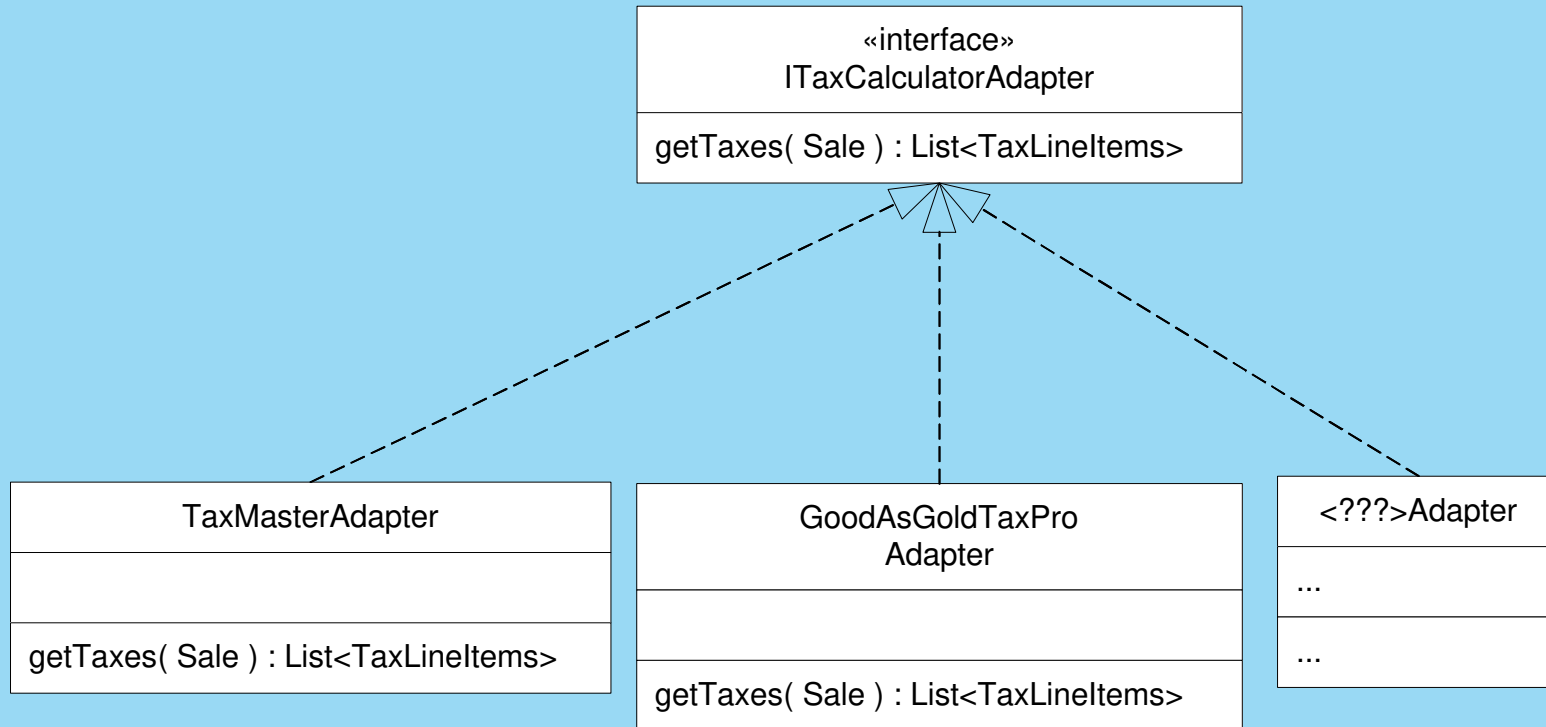


1. Adapter Pattern (GoF)

- **Problem:** How to resolve incompatible interfaces, or provide a stable interface to similar components with different interfaces?
- **Solution:** Convert the original interface of a component into another interface through an intermediate **adapter object**

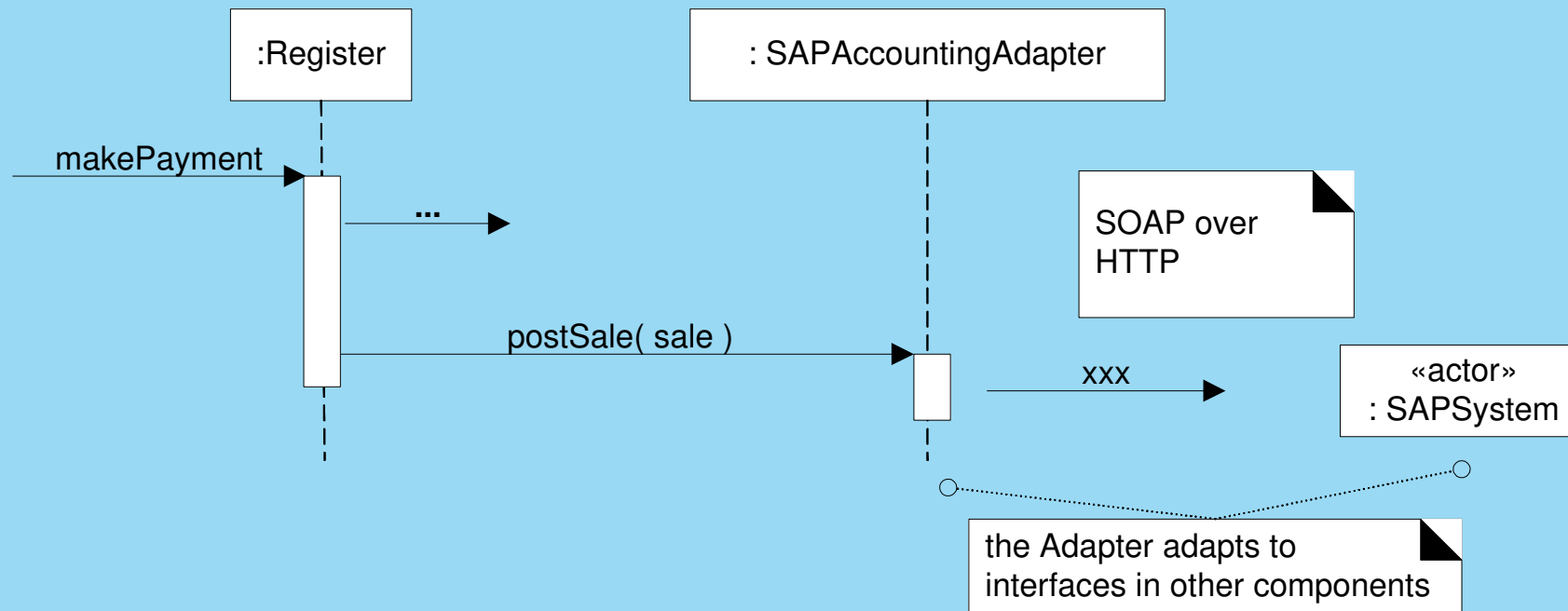


1. Adapter Pattern (GoF)



By Polymorphism, multiple tax calculator adapters have their own similar, but varying behavior for adapting to different external tax calculators.

Another Adapter Example



Question

Which GRASP principles are applied using the Adapter pattern?

a) Low Coupling

b) Indirection

c) High Cohesion

d) Pure Fabrication

Adapter as Specialization of GRASP Principles

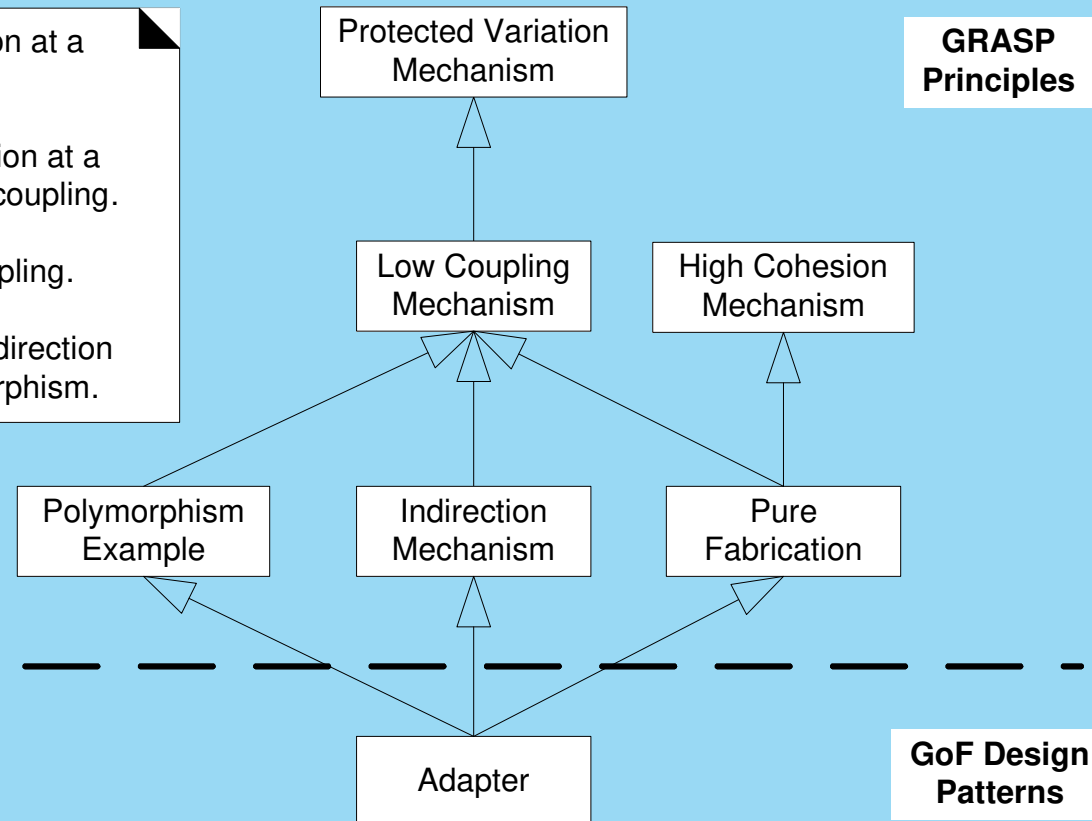
- Adapter supports **Protected Variations** with respect to changing external interfaces or third-party packages through the use of an Indirection object that applies interfaces and **Polymorphism** to achieve **Low Coupling**. At the same time, the Adaptor object itself can be seen as an example of a **Pure Fabrication**.

Low coupling is a way to achieve protection at a variation point.

Polymorphism is a way to achieve protection at a variation point, and a way to achieve low coupling.

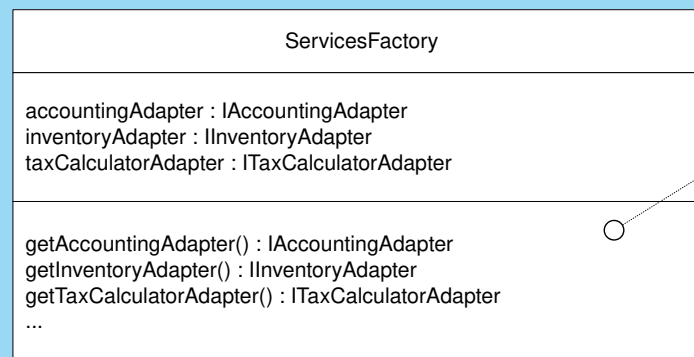
An indirection is a way to achieve low coupling.

The Adapter design pattern is a kind of Indirection and a Pure Fabrication, that uses Polymorphism.



2. Factory Pattern (GoF)

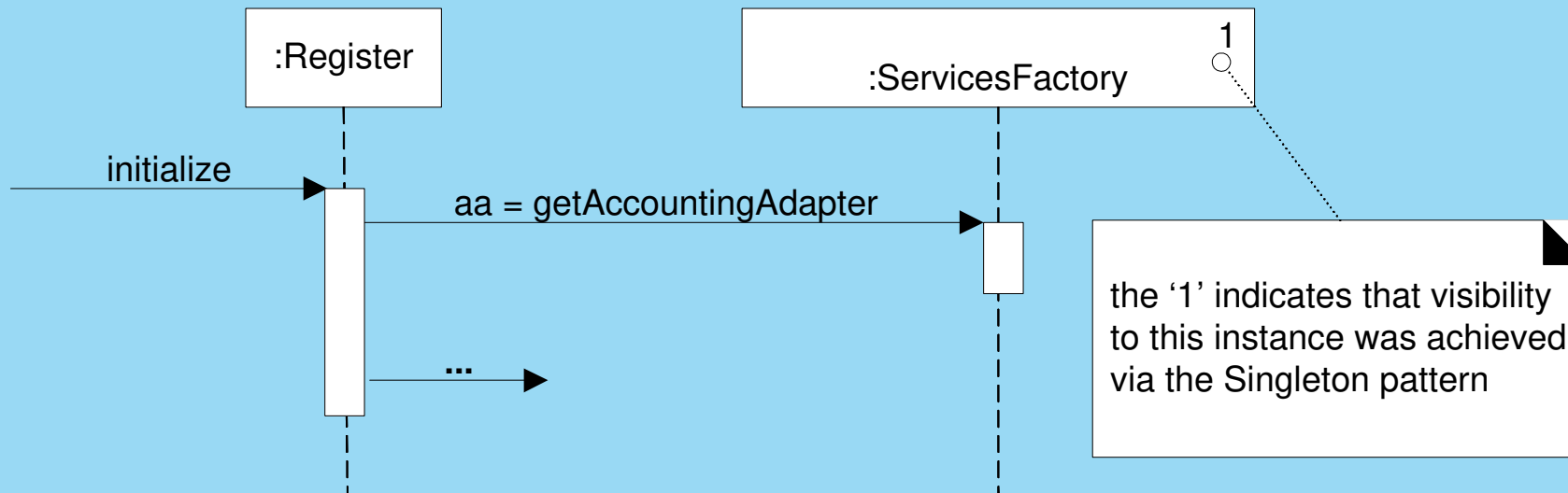
- **Problem:** Who should be responsible for creating objects when there are no special considerations, such as e.g. recommendations provided by the Creator GRASP principle
- **Solution:** Create a „Pure Fabrication“ object called **Factory** that handles the creation
- a.k.a. „Simple Factory“ or „Concrete Factory“
- Simplified version of the GoF „Abstract Factory“ pattern



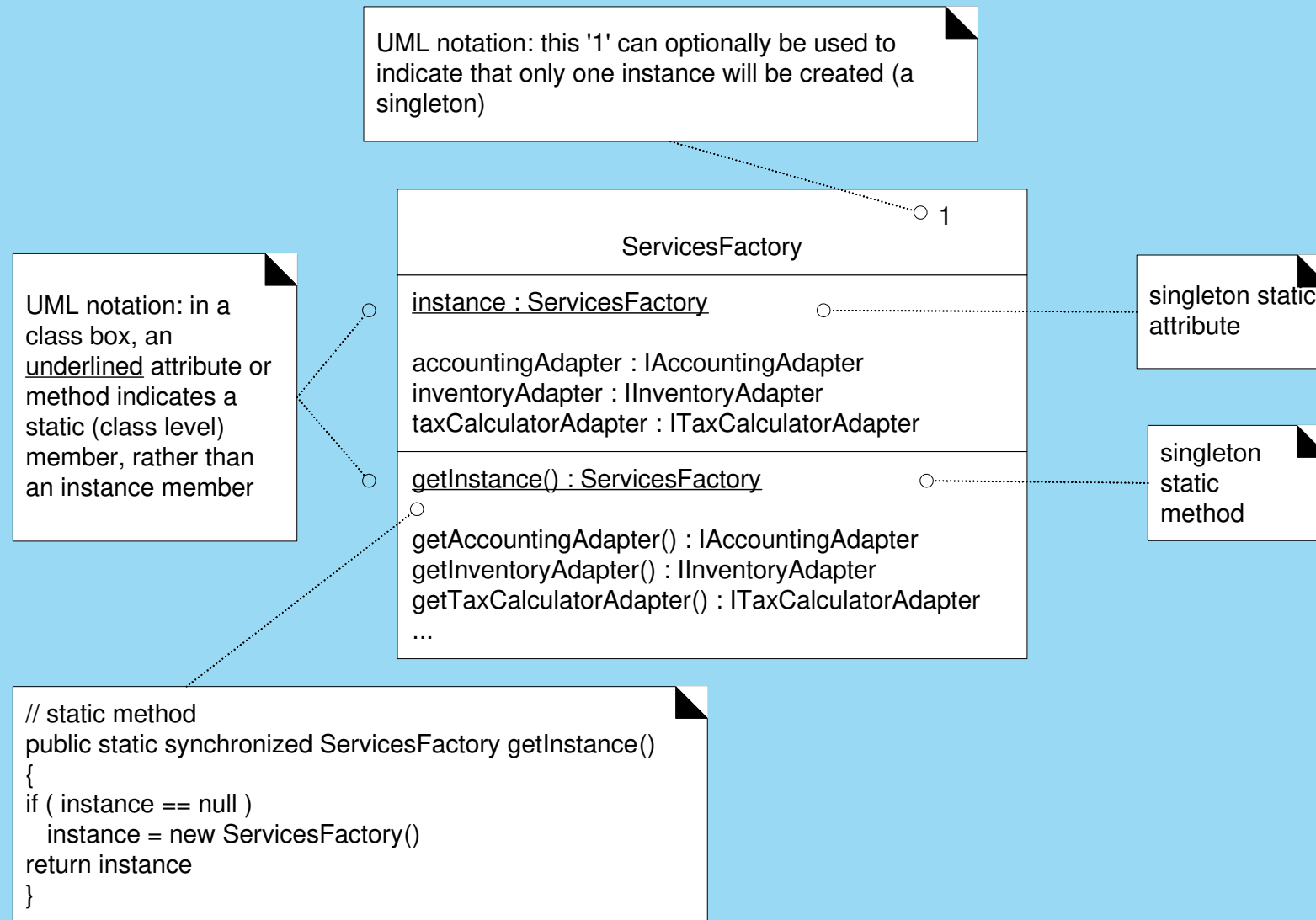
note that the factory methods return objects typed to an interface rather than a class, so that the factory can return any implementation of the interface

3. Singleton Pattern (GoF)

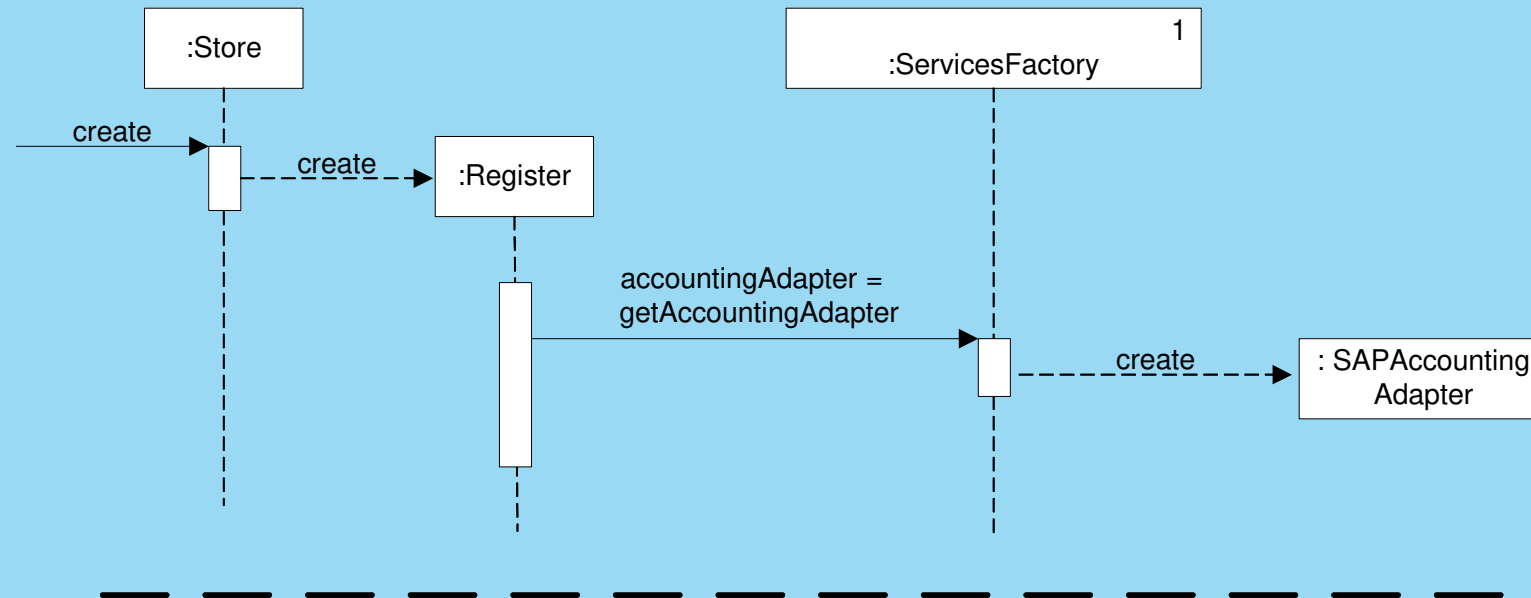
- **Problem:** Exactly one instance of a class is allowed – a „**singleton**“. Objects need a global and single point of access to the instance.
- **Solution:** Define a static method of the class that manages the singleton.



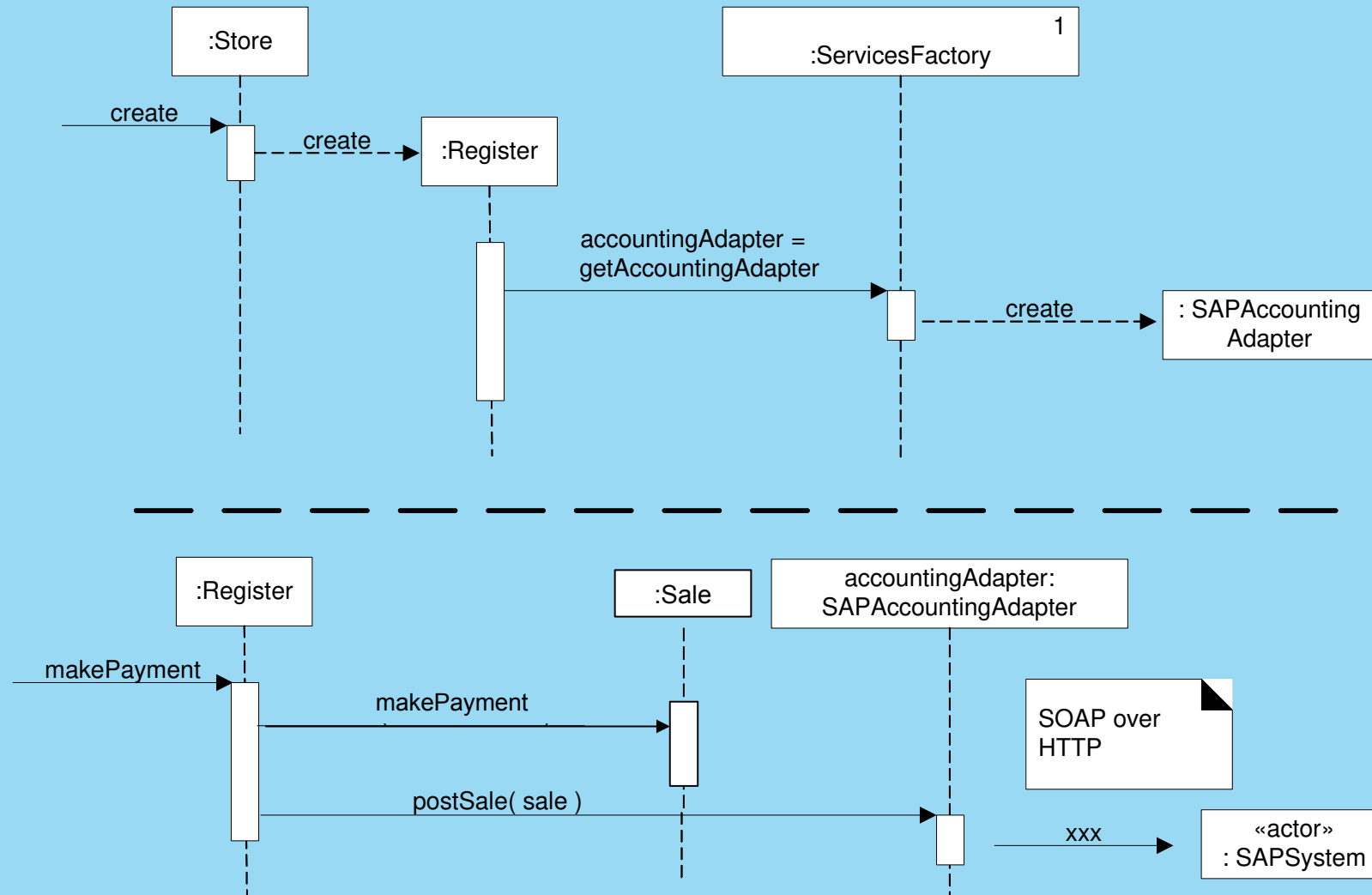
Example: Singleton Pattern



Adapter, Factory, and Singleton Applied



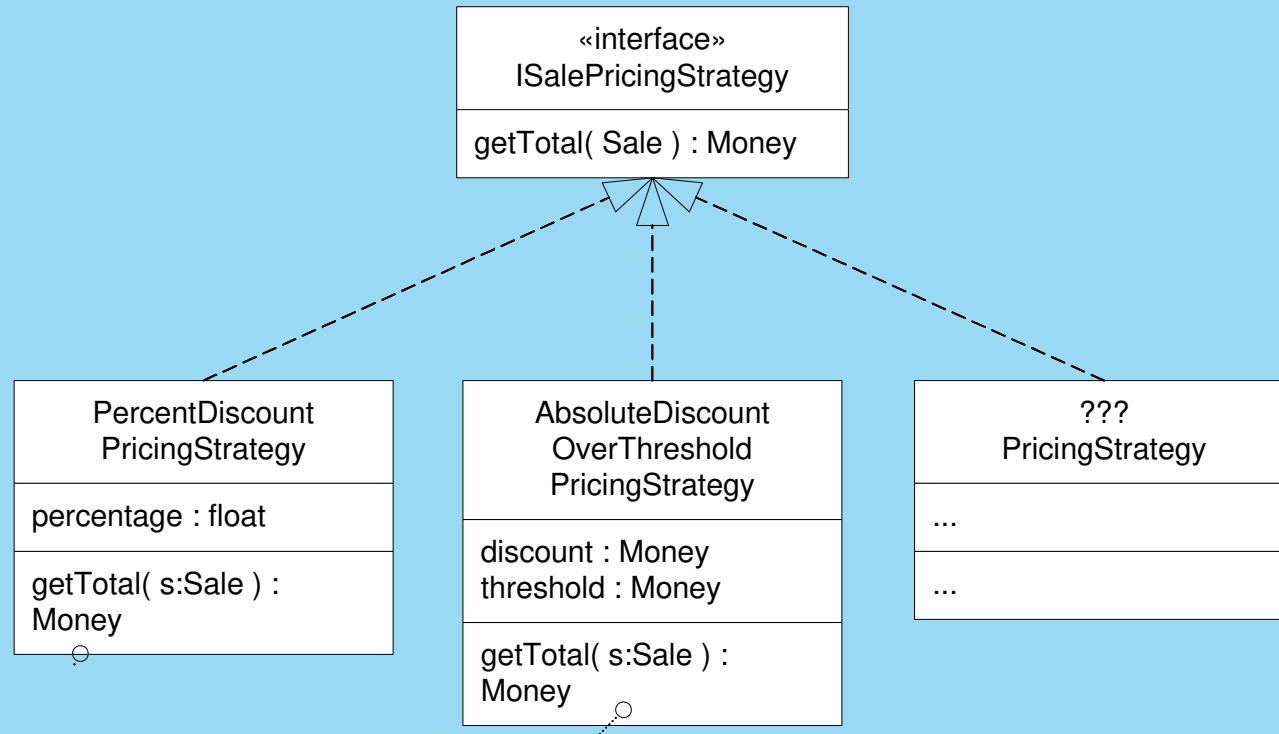
Adapter, Factory, and Singleton Applied



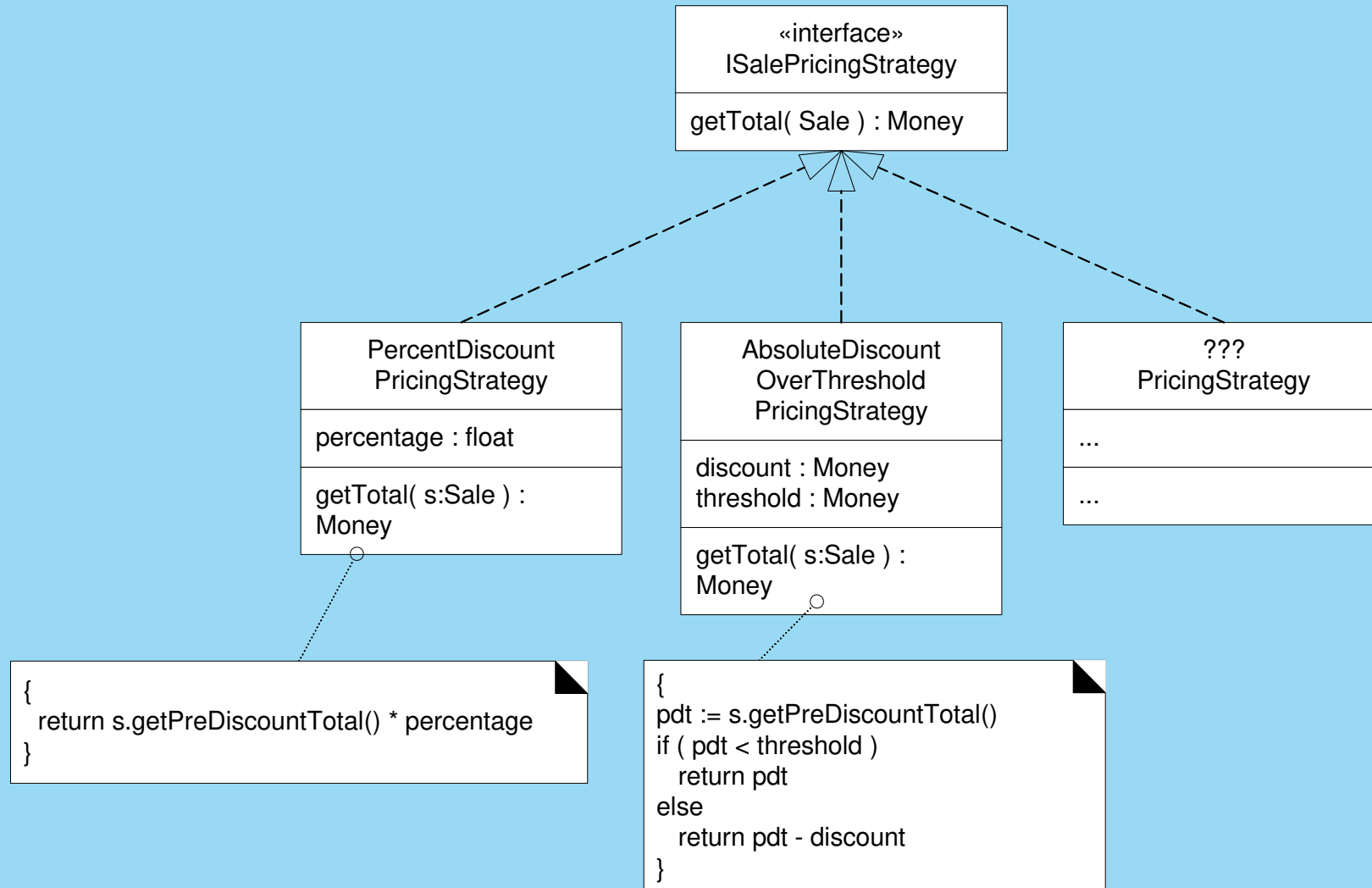
4. Strategy Pattern (GoF)

- **Motivation:** POS System
 - The pricing strategy (which can be seen as a rule, policy, or algorithm) for a Sale can vary. During one period it may be 10% off all sales, later it may be \$10 off if the sale's total is greater than \$200, and myriad other variations. How do we design for these varying pricing algorithms?
- **Problem:**
 - How to design for varying, but related, algorithms or policies?
How to design for the ability to change these algorithms or policies?
- **Solution:**
 - Define each algorithm/policy/strategy in a separate class, with a **common interface**.

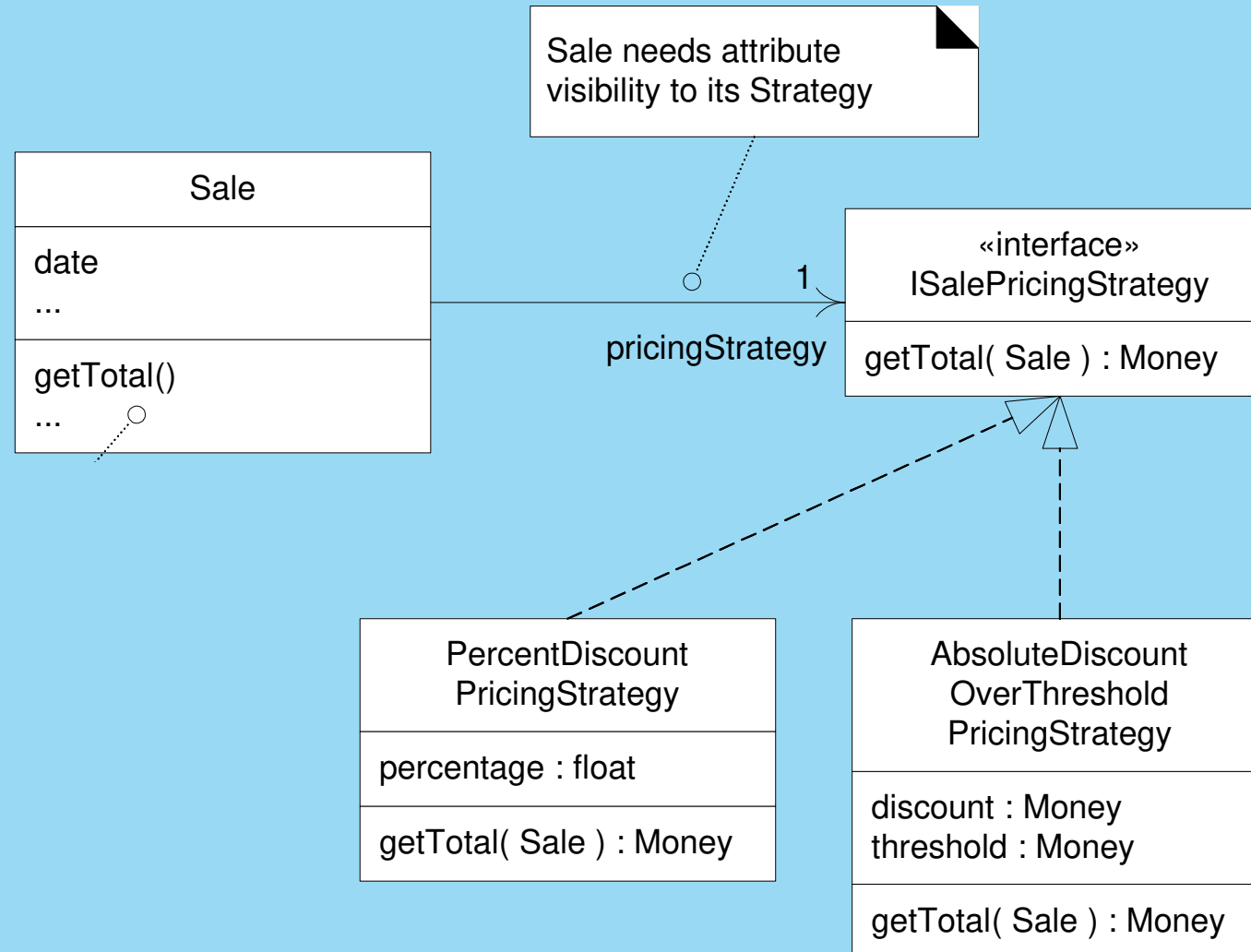
Example: Strategy Pattern (1/6)



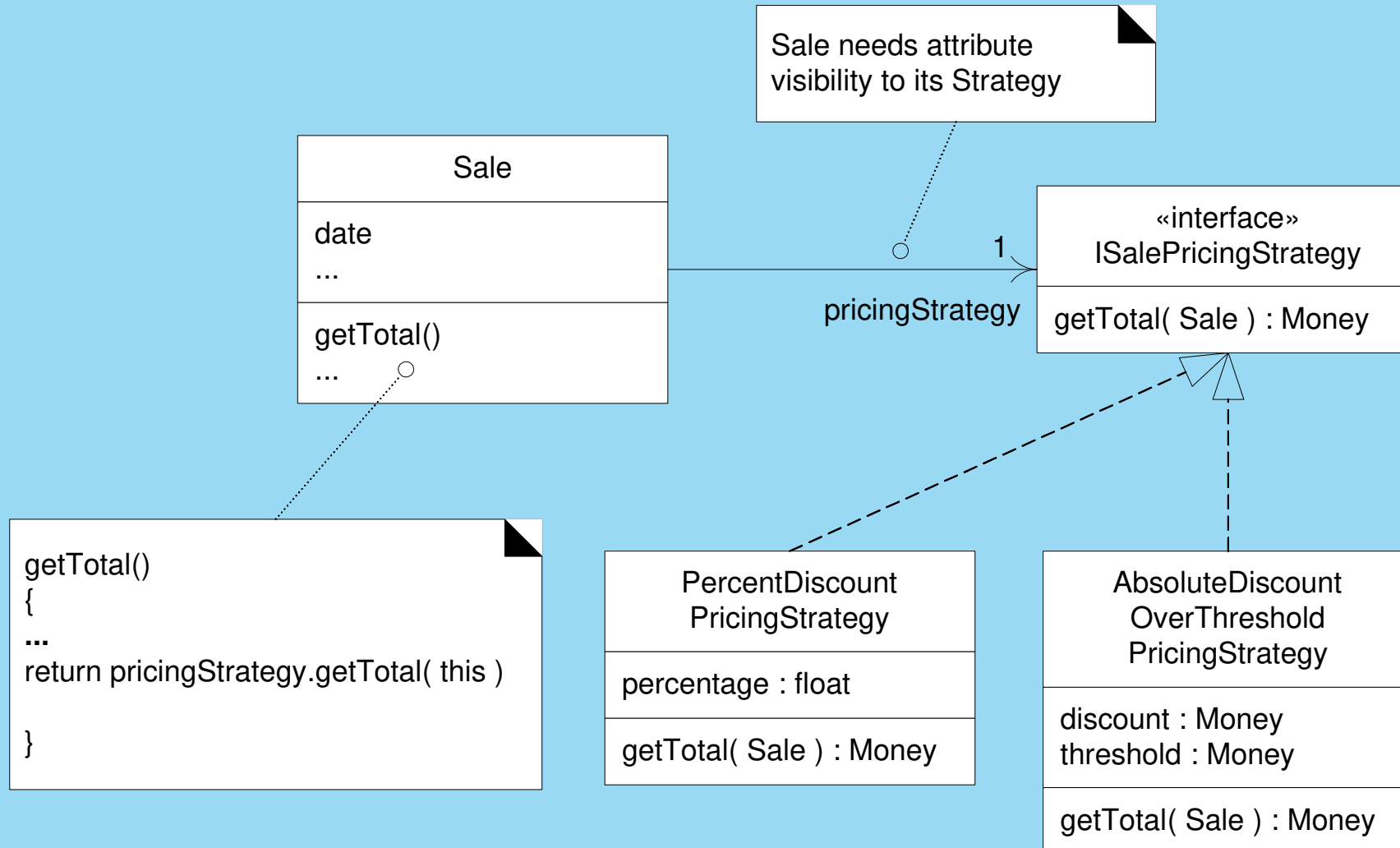
Example: Strategy Pattern (2/6)



Example: Strategy Pattern (3/6)

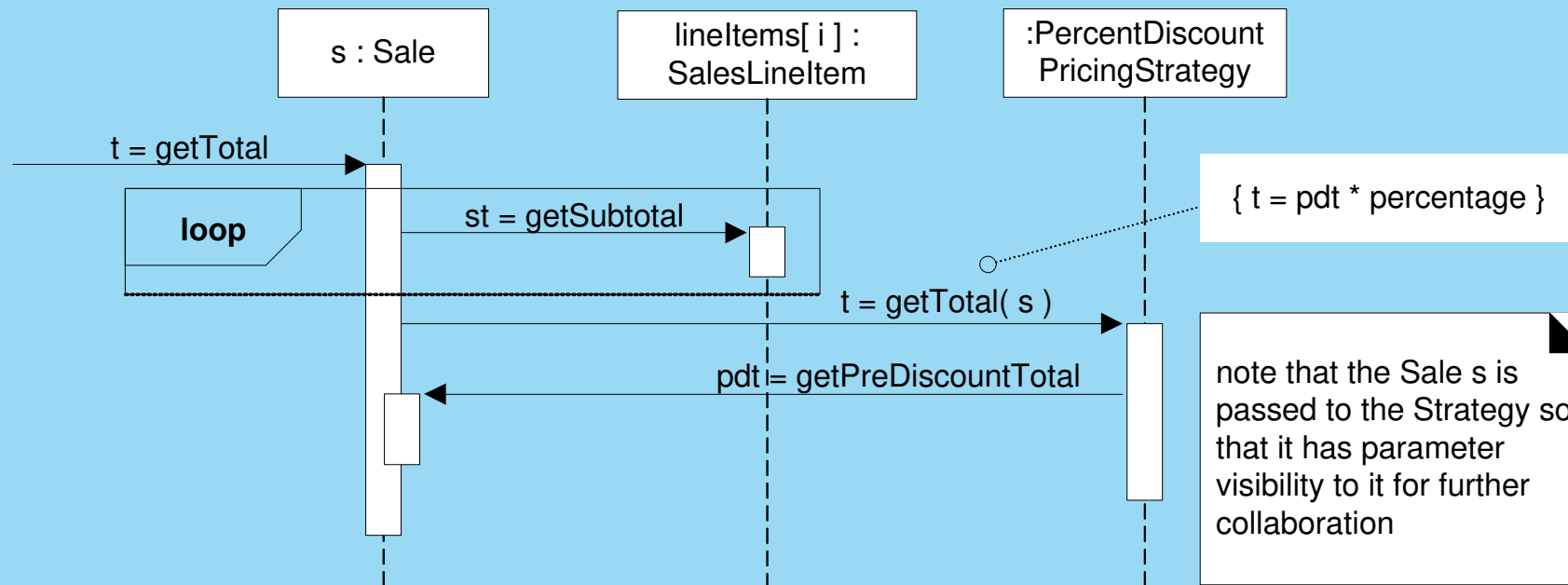


Example: Strategy Pattern (4/6)

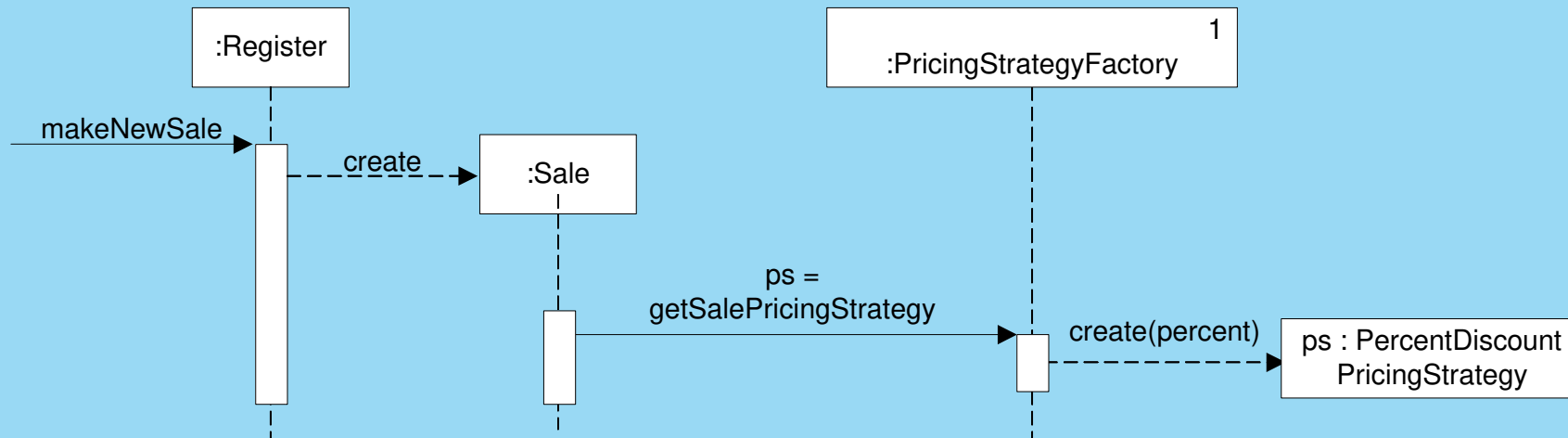


Example: Strategy Pattern (5/6)

- A strategy object is normally attached to a **context object** – the object to which it applies the algorithm. In our example, the context object is a Sale. When a getTotal message is sent to a Sale, it delegates some of the work to its strategy object, as illustrated below:



Example: Strategy Pattern (6/6)



Question

Which GRASP principles are NOT applied in the Strategy pattern?

a) Polymorphism

b) Controller

c) Indirection

d) Pure Fabrication

4. Composite Pattern (GoF)

- **Motivation**

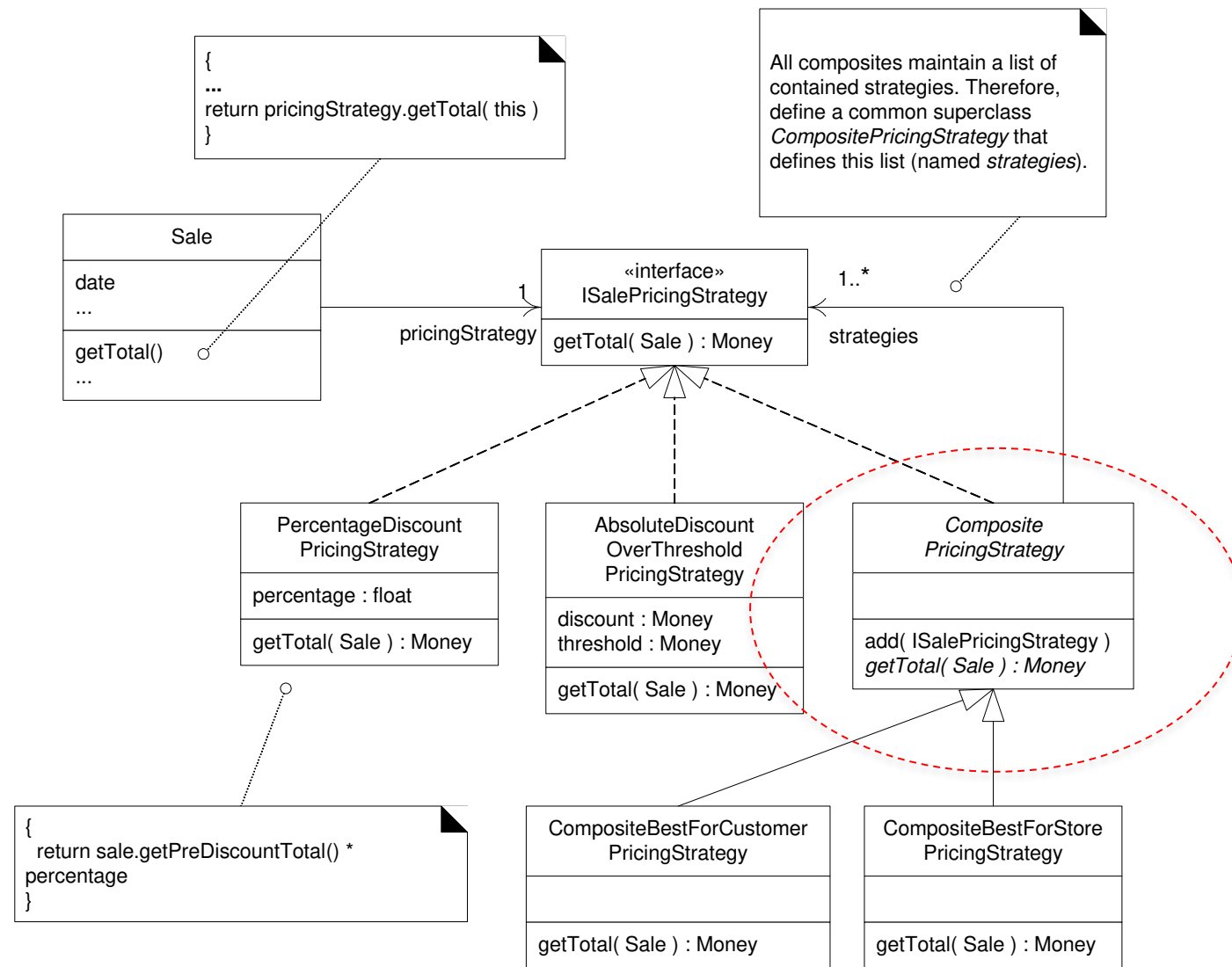
- How do we handle the case of multiple pricing strategies that may be in effect *at the same time*, e.g.:
 - 20% senior discounts policy
 - preferred customer discount of 15% off sales over 400\$
 - buy 1 case of Darjeeling tea, get 15% discount off of everything
- Is there a way to change the design so that the Sale object does not know if it is dealing with one or many pricing strategies?

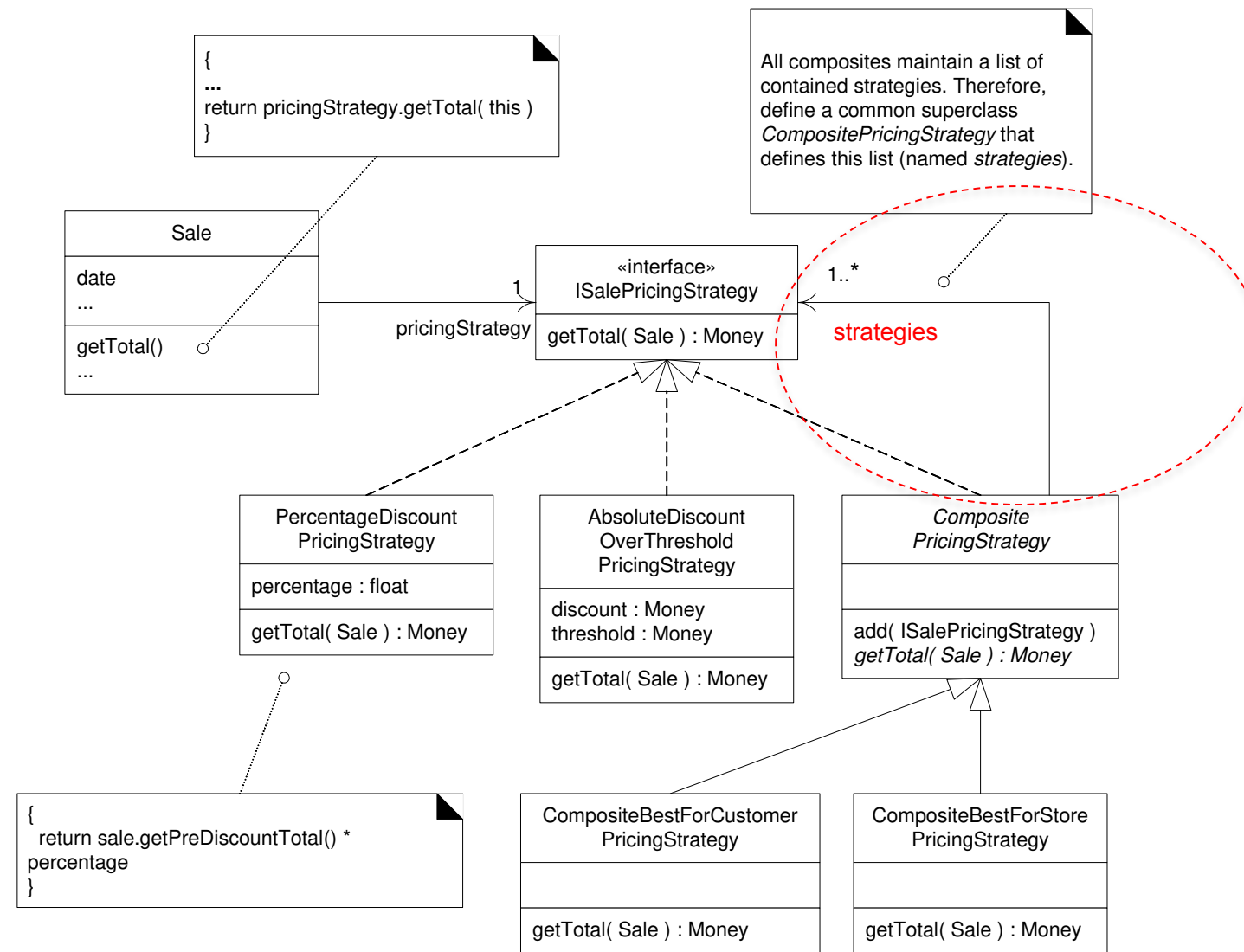
- **Problem:**

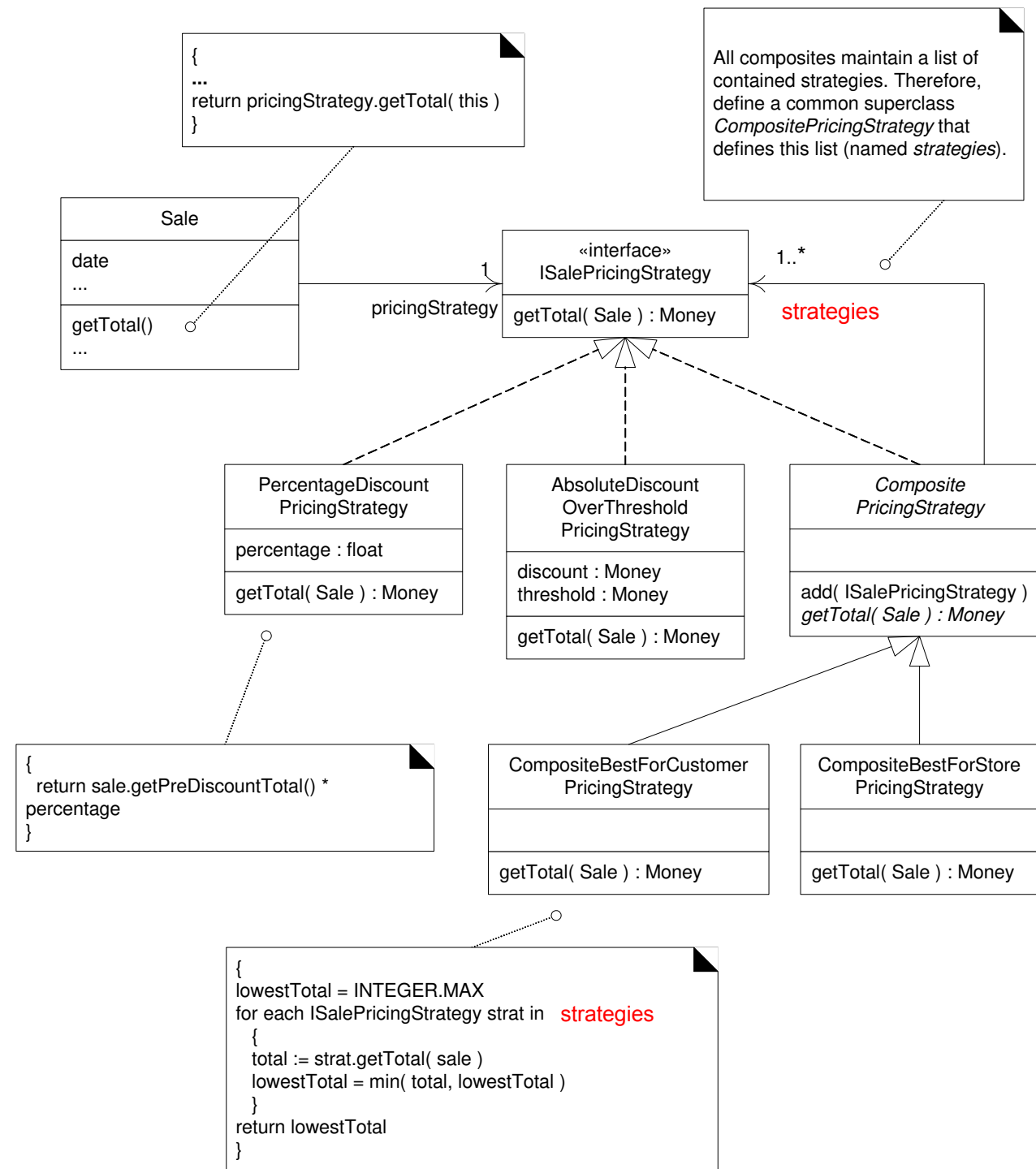
- How to treat a group or composition structure of objects the same way (polymorphically) as a non-composite (atomic) object?

- **Solution:**

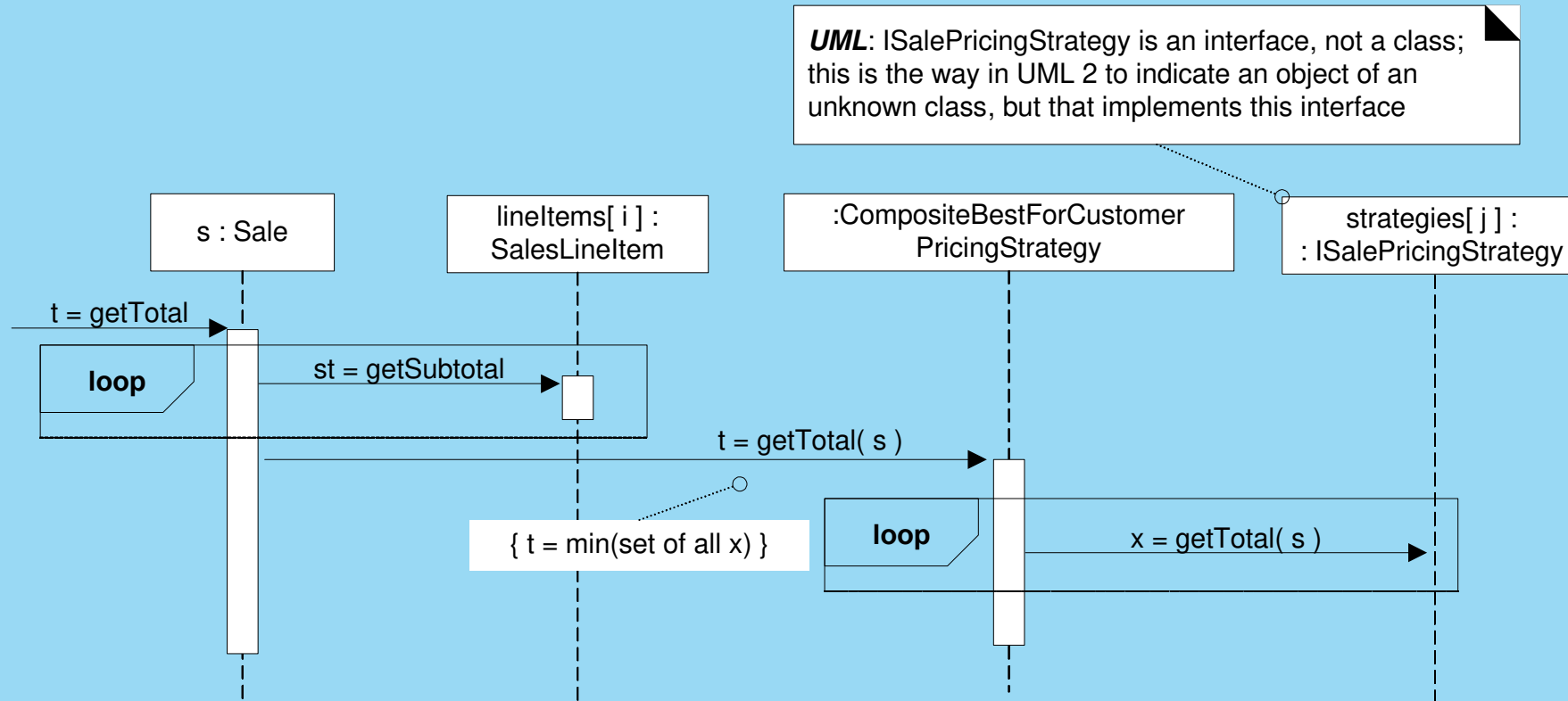
- Define classes for composite and atomic objects so that they implement the **same interface**.







Example: Collaboration with a Composite



the *Sale* object treats a Composite Strategy that contains other strategies just like any other *ISalePricingStrategy*

Question

Which principles are applied using the Composite pattern?

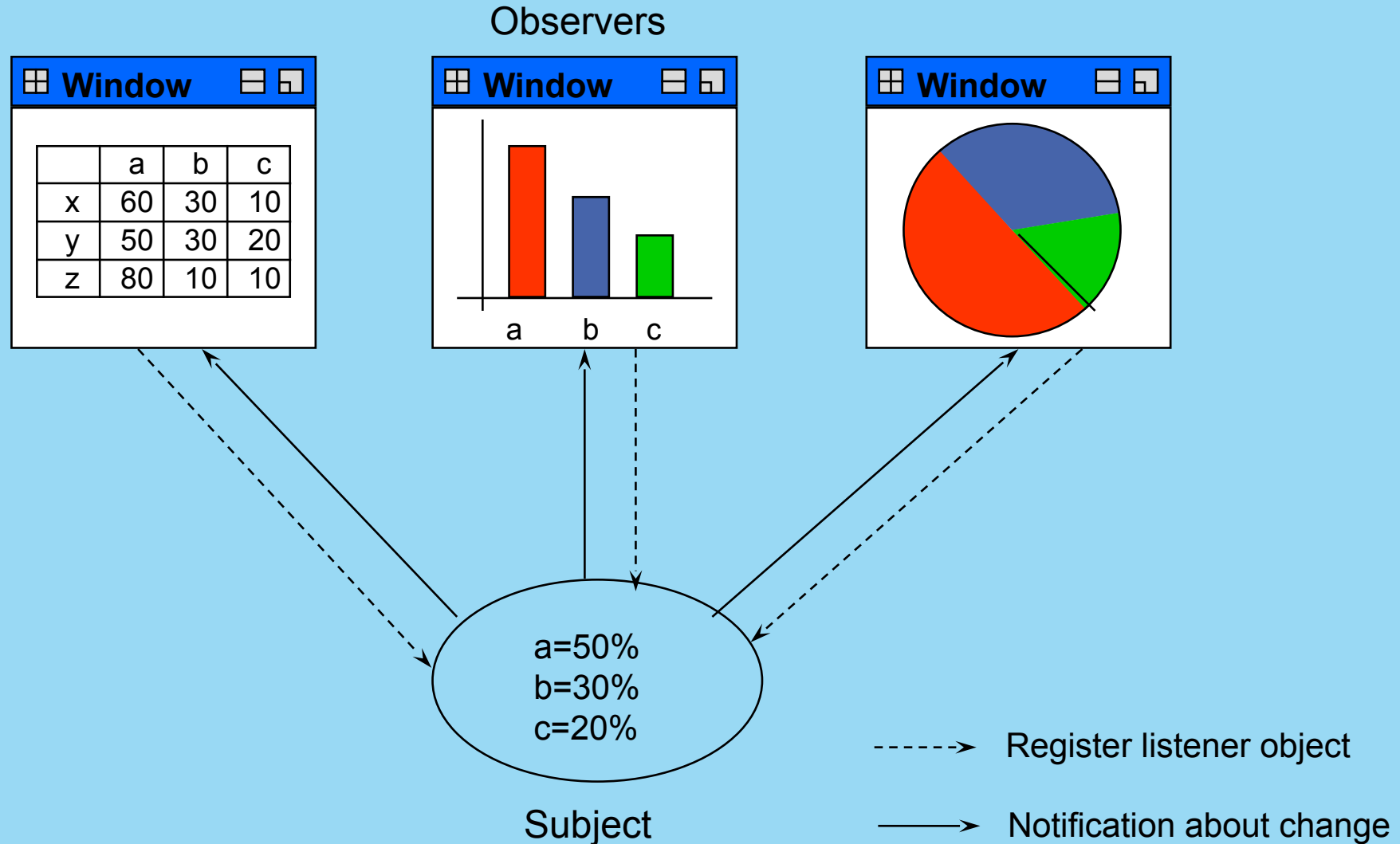
a) GRASP: Protected Variations

b) GRASP: Creator

c) SOLID: Open/Closed Principle

d) GRASP: Polymorphism

Motivation: Observer Pattern (6/6)



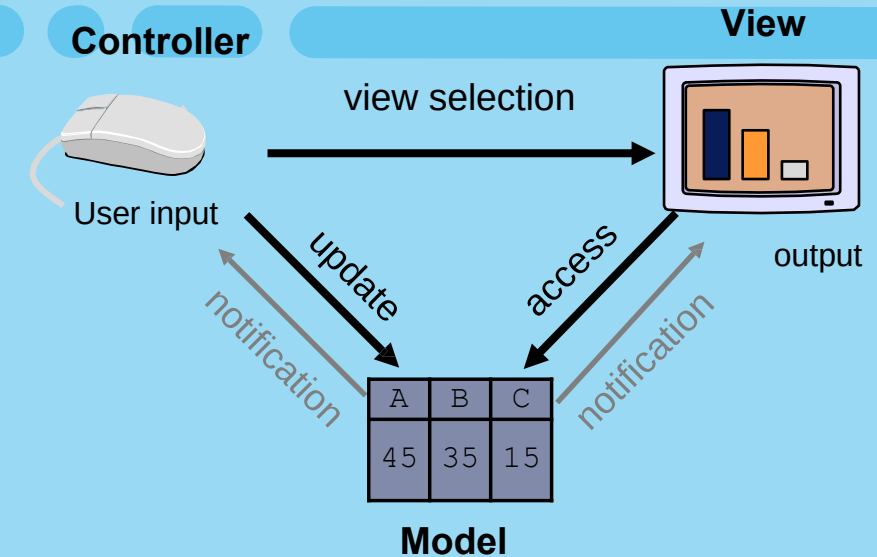
6. Observer Pattern - a.k.a. Publish-Subscribe (GoF)

- **Problem:**
 - Different kinds of subscriber objects are interested in the state changes (or **events**) of a publisher object, and want to react in their own unique way when the publisher generates an event. Moreover, the publisher wants to maintain low coupling to the subscribers.
-
- **Solution:**
 - Define a „**subscriber**“ or „**listener**“ **interface**. Subscribers implement this interface. The publisher can dynamically register subscribers who are interested in an event and notify them when an event occurs.

Model View Controller (MVC) Pattern

Class	Collaborators
Model	- View - Controller
Responsibility - Provides functional core of the application - Registers dependent views and controllers - Notifies dependent components about data changes	

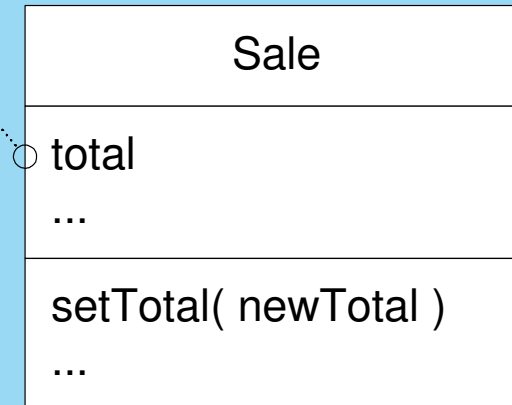
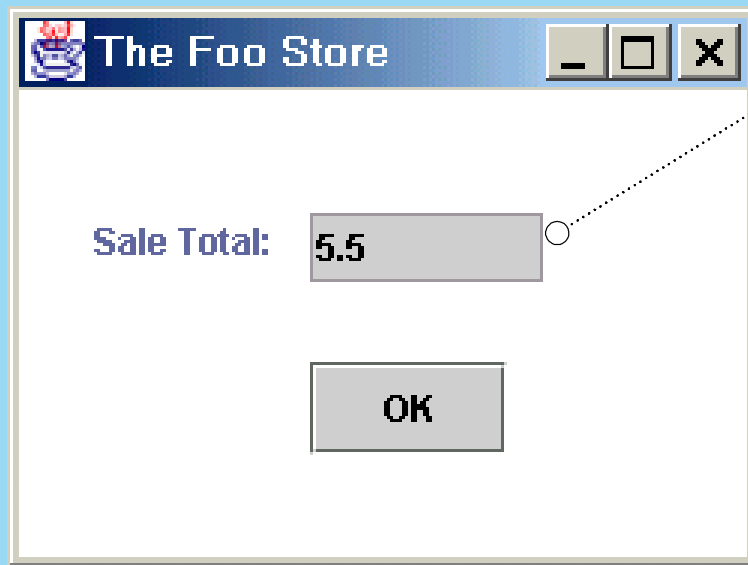
Class	Collaborators
Controller	- View - Model
Responsibility - Accepts user input as events - Translates events to service requests for the model or display requests for the view - Implements the update procedure if required	



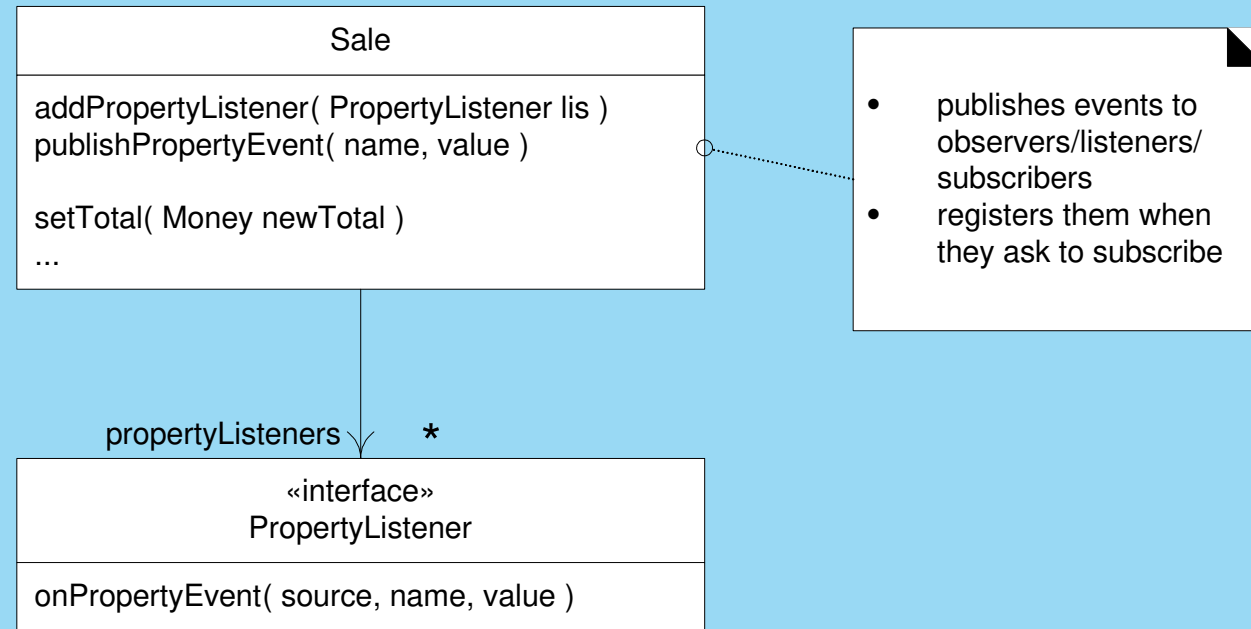
Class	Collaborators
View	- Controller - Model
Responsibility - Creates and initializes its associated controller - Displays information to the user - Implements the update procedure - Retrieves data from model	

Example: Observer Pattern (GoF)

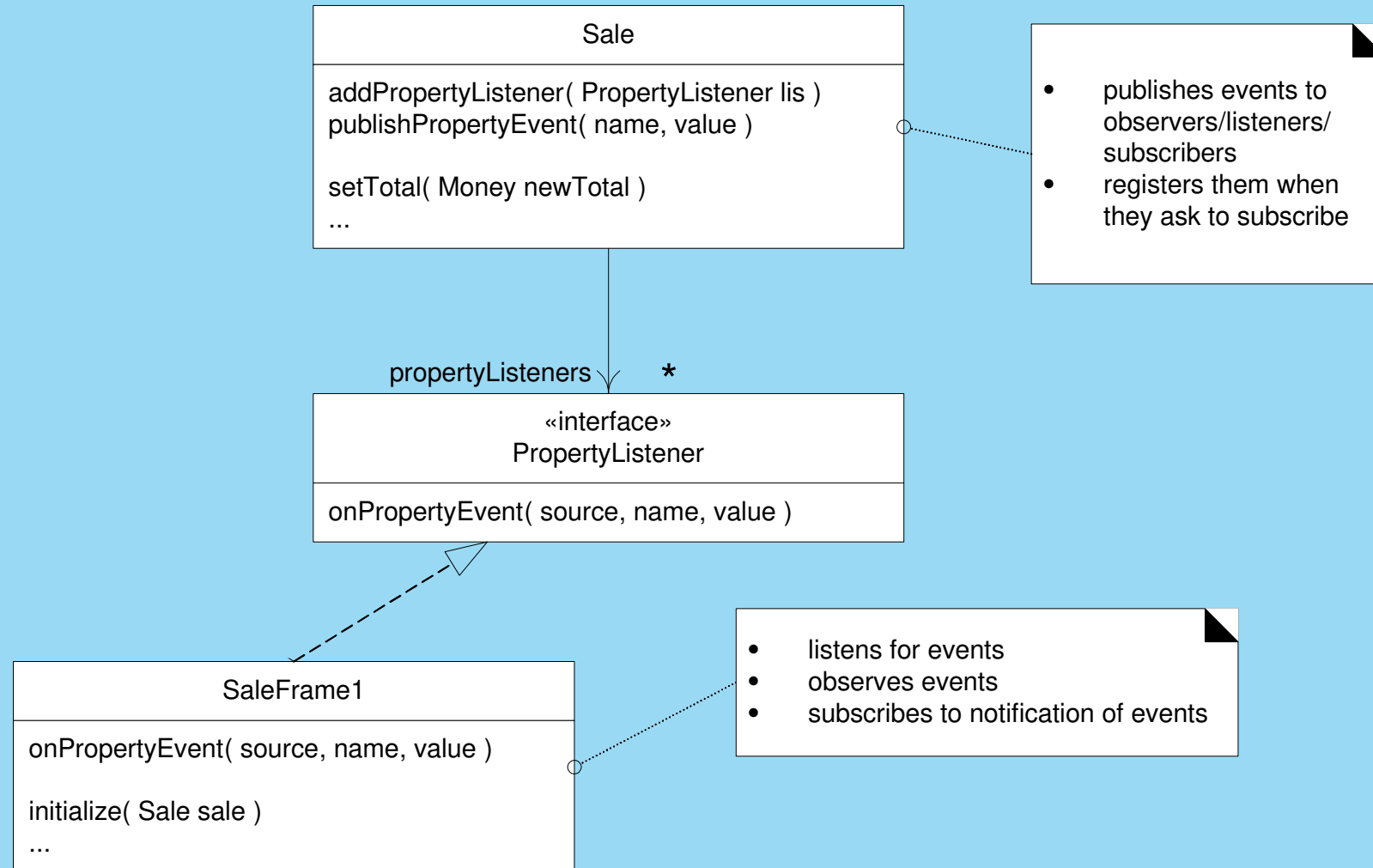
Goal: When the total of the sale changes, refresh the display with the new value



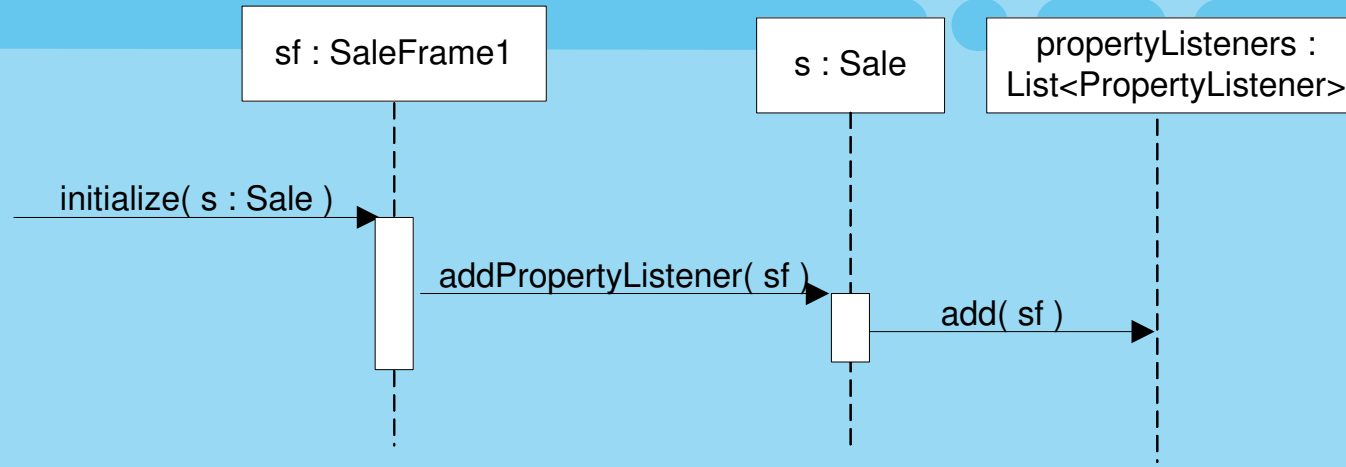
Example: Observer Pattern (GoF)



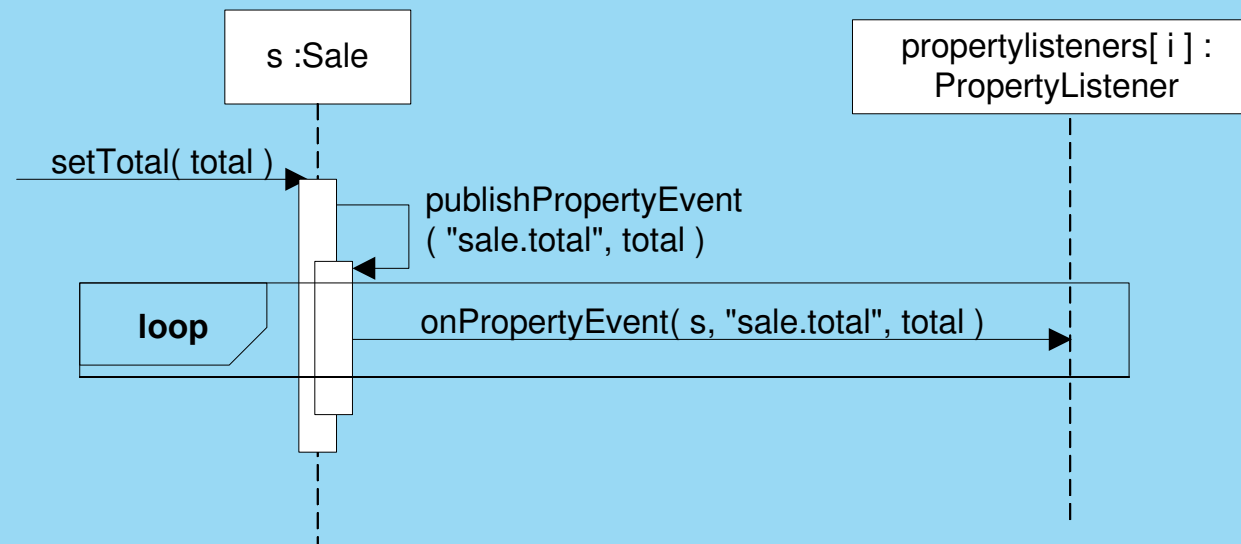
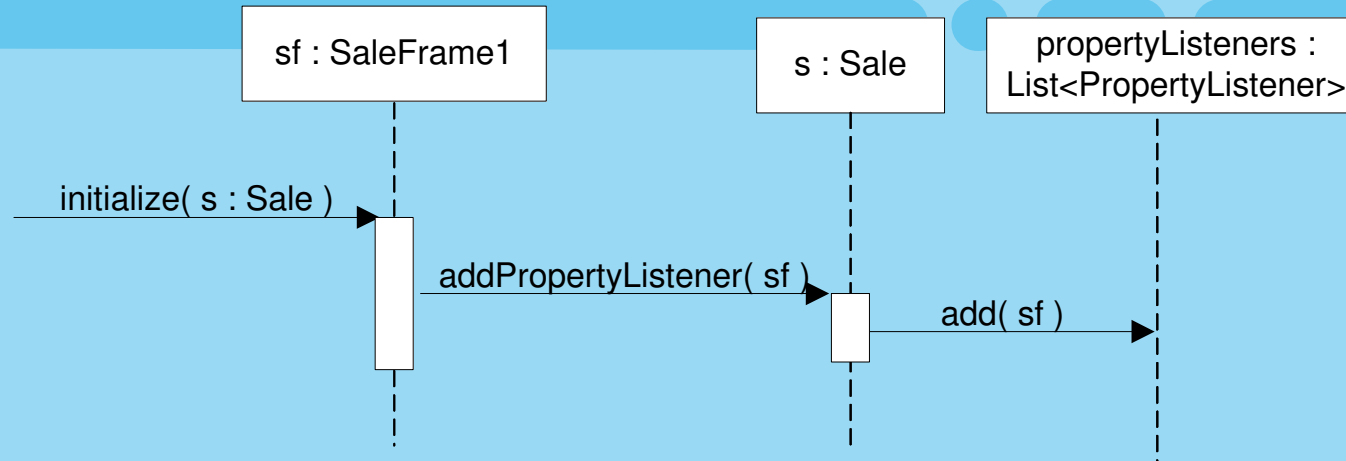
Example: Observer Pattern (GoF)



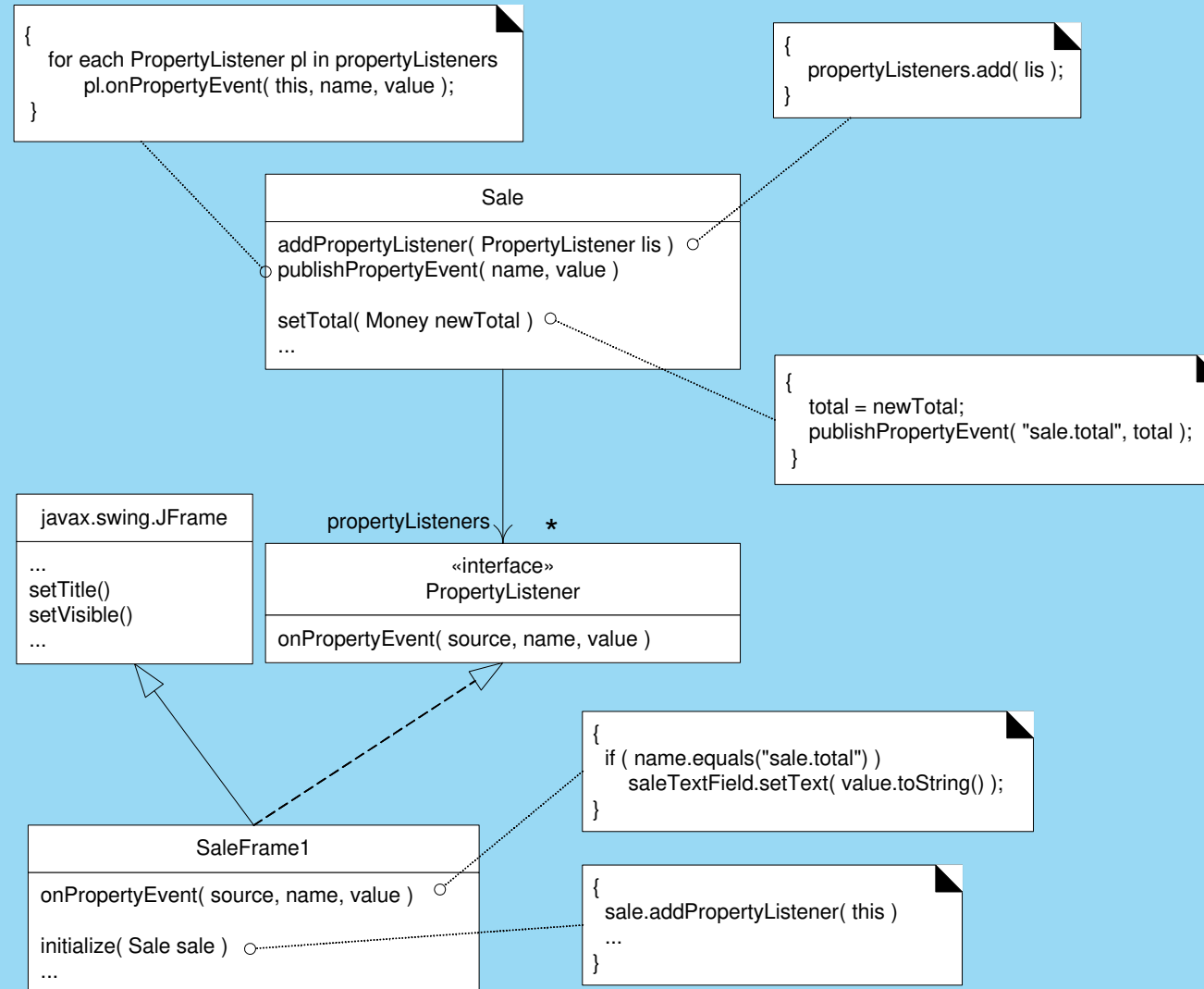
Example: Observer Pattern (GoF)



Example: Observer Pattern (GoF)



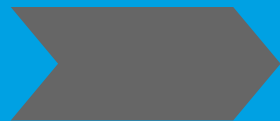
Example: Observer Pattern (GoF)



Design Patterns

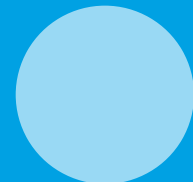
Patterns Revisited

Selected Gang of Four (GoF) Design Patterns



Anti-Patterns

Concluding Remarks



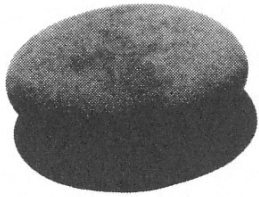
Anti-Patterns [Brown et al.]

- Obviously, **anti-patterns** are the opposite of patterns
 - i.e., they are disproved design and programming habits
 - as such, they have some similarity with **bad smells** (refactorings)
- Three broad categories of patterns exist –
 - **Managerial** referring to managing process and people
 - **Architectural** are about defining the technical strategy
 - **Developmental** deal with coding practices
- Anti-Patterns capture the negative patterns that can cause stagnant development
 - and offer proven solutions for these situations

Example Anti-Pattern

- **Anti-Pattern Name:** Analysis Paralysis (215)
- **Also Known As:** Waterfall, Process Mismatch
- **Most Frequent Scale:** System
- **Refactored Solution Name:** Iterative-Incremental Development
- **Root-Causes:** Pride, Narrow-Mindedness
- **Unbalanced Forces:** Management of Complexity
- **Anecdotal Evidences:**
 - “We need to redo this analysis to make it more object-oriented.”
 - “We need to complete the design before we can start coding.”
- **Some Symptoms:**
 - design and implementation issues are constantly discussed in analysis
 - the analysis phase involves no user interaction
 - GoF patterns are discussed in analysis phase
- **Solution:** Iterative and Incremental development

Caution!



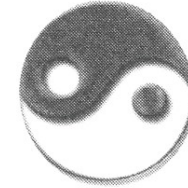
BEGINNER MIND

"I need a pattern for Hello World."



**INTERMEDIATE
MIND**

"Maybe I need a Singleton here."



ZEN MIND

This is a natural place for Decorator.'

Do not "over-engineer" your systems
Sometimes patterns are not required



[Freeman2004]

Recipe for Failure



[https://www.youtube.com/watch?v=nhfh6A_cP_U]

Concluding Remarks

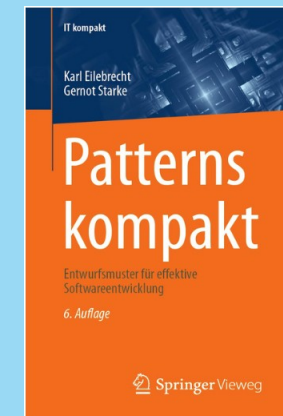
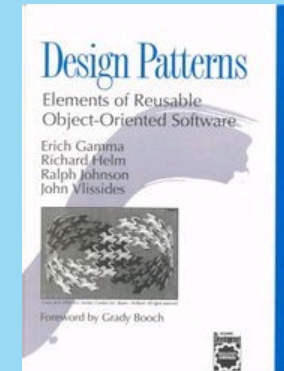
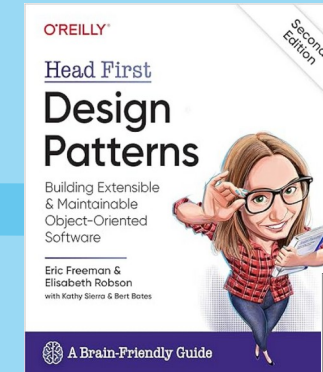
Patterns capture valuable design knowledge

- on various levels and in various areas
 - architectural patterns
 - OO design patterns
 - GRASP...
 - GoF...
 - ...
- help to apply principles of good design
 - may be helpful in Job interviews as well :)



Literature

- **Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software**
Freeman et al., O'Reilly, 2020
- **Design Patterns**
Gamma et al., Addison-Wesley, 1995
- **Patterns kompakt: Entwurfsmuster für effektive Softwareentwicklung (IT kompakt), 6. Auflage**
Karl Eilebrecht, Gernot Starke, Springer Vieweg, 2024



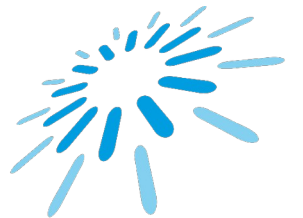
References

- [GoF] E. Gamma, R. Helm, R. Johnson, J. Vlissidies
Design Patterns: Elements of Reusable Object-Oriented Software
Addison-Wesley, 1994.
- [Freeman*2] E. Freeman & E. Freeman
Entwurfsmuster von Kopf bis Fuß
O'Reilly, 2006.
or english: *Original Head First Design Patterns* (2004).
- [Buschmann] F. Buschmann, R. Meunier, H. Rohnert, M. Stal
Pattern-Oriented Software Architecture: A System of Patterns
Wiley, 1996.
- [Fowler] M. Fowler
Patterns of Enterprise Application Architecture
Addison-Wesley, 2003.
- [Brown et al.] W. Brown, R. Malveau, H. McCormick, T. Mowbray
AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis
Wiley, 1998.



Until next time!

Software Architecture and Design Patterns
Prof. Dr. Jóakim von Kistowski



TH Aschaffenburg
university of applied sciences